DON'T WASTE YOUR BACK PRESSURE

PUBLISHED SAT, JAN 17, 2026 BY [MOSS]

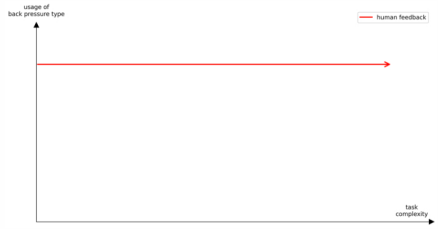
ESTIMATED READING TIME: 4 MIN

Back pressure for agents

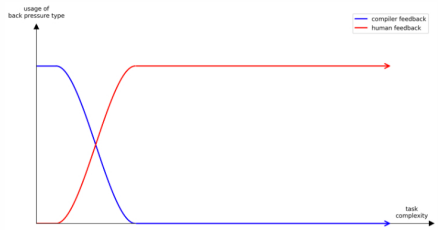
You might notice a pattern in the most successful applications of agents over the last year. Projects that are able to setup structure around the agent itself, to provide it with automated feedback on quality and correctness, have been able to push them to work on longer horizon tasks.

This **back pressure** helps the agent identify mistakes as it progresses and models are now good enough that this feedback can keep them aligned to a task for much longer. As an engineer, this means you can increase your leverage by delegating progressively more complex tasks to agents, while increasing trust that when completed they are at a satisfactory standard.

Imagine for a second if you only gave an agent tools that allow it to edit files. Without a way to interact with a build system the model relies on you for feedback about whether or not the change it made is sensible. This means you spend **your** back pressure (the time you spend giving feedback to agents) on typing a message telling the agent it missed an import. This scales poorly and limits you to working on simple problems.

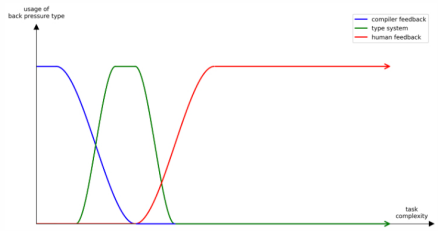


If you're directly responsible for checking each line of code produced is syntactically valid, then that's time taken away from thinking about the larger goals or problems in your software. You're going to struggle to derive more leverage out of agents because you are caught up in trivial changes. If instead you give the agent tools that allow it to run bash commands, it can run a build, read the feedback, and correct itself. You remove yourself from needing to be involved in those tasks and can instead focus on higher complexity tasks.

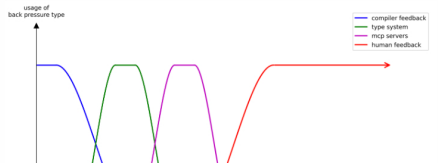


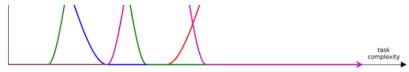
Languages with expressive type systems have been growing in popularity in part because of back pressure. Type systems allow you to describe better contracts in your program. They can let you avoid it from even being possible to represent invalid states in your program. They can help you to identify edge cases that you might not handle. Being able to lean on these features is another form of creating back pressure which you can direct as feedback on changes made by an agent.

Bonus points go to languages that work to produce excellent error messages (think Rust, Elm and even Python). These messages are fed directly back into the LLM so the more guidance or even suggested resolutions the better.



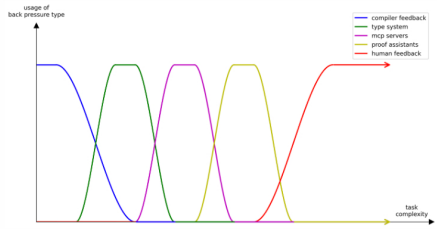
Another example of back pressure is the rapid uptake in people giving agents a way to see rendered pages using MCP servers for Playwright or Chrome DevTools. In either case these tools give the agent a way to be able to make a change and compare an expectation of what it might see in the UI against a result. Attaching these tools mean you remove yourself from needing to keep telling the agent that you're not seeing a UI element load correctly or something isn't centered. Not working on a UI application? Use MCP servers that bridge to LSPs for lints or other feedback.





Even outside of engineering tasks, proof assistants like Lean combined with AI (see recent work on the Erdős Problems which was solved by Kevin Barreto and Liam Price by using Aristotle to formalise a proof written by GPT-5.2 Pro into Lean), randomized fuzzing to evaluate correctness when generating CUDA kernels or logic programming with agents are all powerful combinations because they let you keep pulling the LLM slot machine lever until the result you have can be trusted. I think that the payoff of investing into higher quality testing is growing massively, and an increasing part of engineering will involve designing and building back pressure in order to scale the rate at which contributions from agents can be accepted.

If you're doing spec-driven development and you want the agent to generate a specific API schema, setup automatic generation of documentation based on the OpenAPI schema from your application so the agent can compare the result it produced and what it intended on making. There are many more techniques you can apply similar to this once you recognize the pattern.



In your projects you should think about how you can build back pressure into your workflow and once you have it, you can loop agents until they have stamped out all of the inconsistencies and issues for you. Without it, you're going to be stuck spending your time telling the agent about each mistake it makes yourself.

So next time, think - **are you wasting your back pressure?**

Tagged: ai

© 2026 / POWERED BY HUGO
GHOSTWRITER THEME BY JOLLYGOODTHEMES / PORTED TO HUGO BY JBUB